

PART 2

Architecture

FinVault – MERN Stack Virtual Stock Trading Platform

DOCUMENT INFORMATION

Project Title	FinVault – Virtual Stock Trading Platform
Document Phase	Phase 1: Project Documentation
Part	Part 2 – Architecture
Author	Ganesh Basode
Affiliation	Department of Computer Science & Engineering
Institution	G. Pullaiah College of Engineering and Technology (GPCET)
Date	July 2026
Status	Final Version

1. System Architecture

FinVault implements a standard MERN 3-tier architectural design. This separates concerns, ensuring that the database layer, the business logic layer, and the visual presentation layer are fully decoupled and communicate solely through standard interfaces.

1.1 Three-Tier Model

- **Presentation Layer (Frontend):** Consists of the React SPA built on Vite. It renders UI, executes user interactions, caches state context, and communicates with the server via Axios.
- **Application Layer (Backend):** Composed of Node.js and Express.js. It defines the REST endpoints, executes validation logic, processes user registration/authentication, handles trading operations, and uploads profiles.
- **Data Layer (Database):** Uses MongoDB Atlas as the primary persistence database, storing collections of users, stocks, and transactions via Mongoose schemas.

1.2 Application Communication Flow

When a user interacts with the app (e.g. buying a stock), the React UI dispatches a POST request containing the stock identifier and trade quantity to the Node.js server. The request is processed through the Auth Middleware to extract the user's JWT credentials. The trade controller verifies if the user has sufficient wallet balance and if the stock has adequate quantity. Mongoose updates the User and Stock collections and appends a Transaction log inside a secure atomic operation. Once saved, the server returns the updated user portfolio details in JSON format, prompting the React client to update the UI.

2. Component Design & Folder Modules

2.1 Backend Modules

- **config/db.js:** Implements connection methods to bind Node.js to MongoDB Atlas using Mongoose client drivers.
- **models/:** Holds Mongoose structure definitions representing MongoDB collections (User, Stock, Transaction).
- **routes/:** Binds external URL endpoints to backend controller logic (e.g., auth, trade, stock, admin).
- **controllers/:** Executes the actual DB queries and business rules (e.g., matching trades, user verification).
- **middleware/:** Integrates interceptors (e.g., checking tokens via authMiddleware, multer upload settings).

2.2 Frontend Modules

- **context/AuthContext.jsx:** Context provider storing the user's logged-in status, authentication JWT, profile data, and wallet balances globally.
- **pages/:** Views containing dashboard lists, watchlist grids, profile sheets, and admin configurations.
- **components/:** Modular widgets like Navbar, Sidebar, StockChart, and Transaction tables.
- **api/:** Setup scripts establishing the API instance using Axios with authentication headers automatically attached.

3. Database Schema & Data Models

3.1 User Schema

Defines the fields mapping user accounts, assets portfolio, and watchlists.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  isAdmin: { type: Boolean, default: false },
  balance: { type: Number, default: 100000 },
  profileImage: { type: String, default: "" },
  portfolio: [
    {
      stockId: { type: mongoose.Schema.Types.ObjectId, ref: "Stock" },
      quantity: { type: Number, default: 0 },
      averagePrice: { type: Number, default: 0 }
    }
  ],
  watchlist: [{ type: mongoose.Schema.Types.ObjectId, ref: "Stock" }]
});
```

3.2 Stock Schema

Represents individual listed stocks, available mock quantity, and historical values.

```
const stockSchema = new mongoose.Schema(
{
  symbol: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  price: { type: Number, required: true },
  quantityAvailable: { type: Number, default: 1000 },
  priceHistory: { type: [Number], default: [] }
},
{ timestamps: true }
);
```

3.3 Transaction Schema

Logs historical trades executed by users on the platform.

```
const transactionSchema = new mongoose.Schema(
{
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  stockId: { type: mongoose.Schema.Types.ObjectId, ref: "Stock" },
  type: { type: String, enum: ["BUY", "SELL"] },
  quantity: Number,
  price: Number,
  total: Number
},
{ timestamps: true }
);
```